

# DW and EDA Final Project Presentation: YouTube Trending Stats

By Gavin Binder and Aidan Sieger

# Topic and Motivation

- ▶ For this project, we chose to explore YouTube trending videos. YouTube is one of the most popular websites on the planet, with millions of daily users and billions of videos generating massive amounts of data.
- ▶ YouTube creates trending charts, which aim to highlight fast growing videos from diverse creators and topics. Most notably, for a video to become trending it must be new and have a fast-growing view count.
- ▶ Due to this, the trending charts are a great source of information on what the "hottest" topics were at the time.

# Dataset

- ▶ For our dataset, we will be using the YouTube Trending Videos Stats 2026, from Kaggle, created by Shul Ilya. This dataset has data about trending videos from 2007 to 2026.
- ▶ The dataset has 13 variables, such as: video ID, trending date, video title, channel title, view count, and like count.
- ▶ The dataset includes over 178,400 rows from 11 countries, separated by file. The countries include the United States, Canada, and Japan to name a few.
- ▶ This dataset is sourced by the authors using the YouTube Data API v3 and the Return YouTube Dislike API.

# Preliminary Data Quality Assessment

- ▶ Uses the official YouTube API to collect its data.
  - This API is directly from YouTube, so guarantees accuracy.
- ▶ Dislikes and trending date have no variability
- ▶ Each country is separated by file.
- ▶ No NA values outside of description, but these NA were determined to mean that there was no description, not that the value was missing.
- ▶ Major issue: Each country was collected by the author at a different time. This means that some videos that trended in multiple countries have different view counts due to timing (note: columns like views is not a per country, it is total for the video) therefore, we use the maximum view/like/comment counts for a video among different countries to get the latest snapshot.

# Related Work

- ▶ We have reviewed multiple sources dealing with the integration of LLMs into data wrangling and exploratory data analysis processes. Some of those sources include:
  - Can LLMs Clean Up Your Mess? A Survey of Application-Ready Data Preparation with LLMs
  - Large Language Models as Data Preprocessors
  - Can language models automate data wrangling?
- ▶ Each source looks into using LLMs in data cleaning, data integration, and data enrichment. While each source points out that LLMs have strong benefits to their use in data wrangling processes, each also comes with limitations. For the most part, the main limitations said by the papers is that LLMs have high computational cost and can often hallucinate.

# Pre-LLM Data Wrangling

- ▶ We did the following data wrangling steps before using LLM integration:
  - Loaded each file, added a country column to them, then combined into a single YT\_trending data frame..
  - Removed the dislikes and trending\_date column due to them having 0 variability (no variability makes it a mostly useless variable)
  - We also conducted some feature engineering to create columns for has\_tags, num\_of\_tags, has\_desc, desc\_length, title\_length, and channel\_title length.
  - Created a new data frame that has only one entry per video (instead of an entry for each country it trended in)

# LLM Integration Approach Part 1

1. We have implemented a local LLM solution that allows us to directly query from our R Markdown file.
2. For this, we use Ollama in conjunction with the Ellmer library.
  - Ollama is an application designed to allow you to run LLMs locally rather than through a web interface or API. It gives you access to some of the latest models such as googles Gemma and Metas Llama.
  - Using the Ellmer package, designed by a team at Posit, you can very easily integrate LLMs directly into your R Markdown file.
3. Running the model and querying it is simple:
  - First, download the model through Ollama
  - Next, we create an object using the `chat_ollama()` function.
  - From there, you can query by simply doing `chat$chat("Hello!")`

# LLM Integration Approach Part 2

1. Unlike just running the model, making it useful and integrating it into our process was less simple.
2. For the obvious reason of there being 178,399 rows and 16 columns, feeding the LLM our whole dataset is impossible.
3. Instead, we had to feed the LLM summary statistics and sample rows so that it could get a general idea of the data.
  - We included row/column counts, column names, mean, standard deviation, min, and max values, categorical counts, and for non-categorical features, a unique count.



# LLM Integration Approach Part 3

- Below is the system prompt given to the LLM (Note: this is just a snippet as the full thing is too large):

```
You are a data analyst assistant.
Your task is to suggest useful visualizations and insights

Dataset overview:
Rows: 178399
Columns: 16
Column names: video_id, title, channel_title, views, likes, publish_time, category_id, comments, channel_id,
country, has_tags, num_of_tags, has_desc, desc_length, title_length, channel_title_length
Summary of numerical columns:
# A tibble: 1 x 36
  views_mean views_sd views_min views_max likes_mean likes_sd likes_min likes_max comments_mean comments_sd
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 1444214. 10814645. 0 1105386196 36071. 126959. 0 8968181 1253. 9773.
  comments_min comments_max has_tags_mean has_tags_sd has_tags_min has_tags_max num_of_tags_mean num_of_tags_sd
  <dbl> <dbl> <dbl> <dbl> <int> <int> <dbl> <dbl>
1 0 3391804 0.742 0.437 0 1 13.0 12.3
  num_of_tags_min num_of_tags_max has_desc_mean has_desc_sd has_desc_min has_desc_max desc_length_mean
  <int> <int> <dbl> <dbl> <int> <int> <dbl>
1 0 125 0.875 0.331 0 1 810.
  desc_length_sd desc_length_min desc_length_max title_length_mean title_length_sd title_length_min
  <dbl> <int> <int> <dbl> <dbl> <int>
1 954. 0 5000 52.5 23.8 1
  title_length_max channel_title_length_mean channel_title_length_sd channel_title_length_min
  <int> <dbl> <dbl> <int>
1 105 11.0 5.73 1
  channel_title_length_max
  <int>
1 56
```

```
Summary of category_id:
# A tibble: 15 x 2
  category_id count
  <dbl> <int>
1 1 9342
2 2 759
3 10 44653
4 15 382
5 17 1923
6 19 181
7 20 76189
8 22 11570
9 23 894
10 24 28999
11 25 1393
12 26 862
13 27 439
14 28 790
15 29 23
Summary of countries:
# A tibble: 11 x 2
  country count
  <chr> <int>
1 Brazil 16200
2 Canada 16200
3 France 16200
4 Germany 16200
5 India 16199
6 Japan 16200
7 Mexico 16000
8 Russia 16200
9 South Korea 16200
10 United Kingdom 16400
11 United States 16400
Unique values for character columns (non chategorical)
# A tibble: 1 x 5
  video_id title channel_title channel_id country
  <int> <int> <int> <int> <int>
1 133188 128651 13077 13089 11
```

# LLM Integration Approach Part 4

1. With this, we could finally begin querying the LLM.
2. The main query we did was quite simple:
  - `chat$chat("Is there any necessary data cleaning, data integration, or data enrichment that needs to be done for this dataset?")`
  - Note: The following is a summary of the recommendations given by the LLM. The following slide is a small snippet of the LLMs output.
    - Check/verify `publish_time` format, check/verify no missing values, handle outliers, confirm `category_id` consistency, string cleaning (whitespace handling and character encoding), and country naming consistency.
    - External data integration (channel stats, geographic stats)
    - Text analysis, sentiment analysis, keyword extraction, time-based features (day of week, video age), ratios (engagement rate, interaction rate)

# LLM Integration Approach Part 5

## 1. A snippet of the LLMs response:

Okay, let's break down potential data cleaning, integration, and enrichment steps that would be beneficial for this dataset. Given the description and summary information, here's a prioritized list with explanations:

**\*\*1. Data Cleaning (Highest Priority)\*\***

- \* **\*\*`publish\_time` Data Type Correction:\*\*** The summary mentions `publish\_time` is a `dtm` (date-time) object. It's *critical* to verify this. Often, date/time columns are imported as strings. If it's a string, you *must* convert it to a proper date/time format for analysis. Incorrect formats will wreck time-series analysis or any date-based calculations.
- \* **\*\*Missing Values:\*\*** While summaries don't explicitly mention missing values, it's a standard practice to check *every* numerical column (views, likes, comments, num\_of\_tags, desc\_length, title\_length, channel\_title\_length) for `NA` values. Handle them appropriately (imputation with mean/median/mode, or removal if the missingness is substantial and biased).
- \* **\*\*Outlier Handling for `views`:\*\*** The `views\_sd` is *huge* compared to `views\_mean`. This indicates a very wide distribution and likely contains outliers. Extreme values in `views` can skew any analysis. Consider:
  - \* Visual inspection (histograms, boxplots) to identify outliers.
  - \* Winsorizing or trimming extreme values to reduce their impact. (Be careful - might be legitimate viral videos!)
- \* **\*\*`category\_id` Consistency:\*\*** Look for any unexpected or invalid `category\_id` values. Occasionally there might be errors in data entry (e.g., a value that doesn't belong to the established categories).
- \* **\*\*String Column Cleaning (Title, Channel Title, Description):\*\***
  - \* **\*\*Whitespace:\*\*** Trim leading and trailing whitespace from `title`, `channel\_title`, and `desc`.
  - \* **\*\*Character Encoding:\*\*** Ensure consistent character encoding (e.g., UTF-8) to handle special characters correctly. Problematic encodings can lead to errors in text analysis.
  - \* **\*\*HTML/Special Characters:\*\*** The descriptions might contain HTML tags or other special characters that need to be removed or replaced.
- \* **\*\*`country` Consistency:\*\*** While the summary shows a list of countries, ensure consistency in country names (e.g., "United States" vs. "USA"). Standardize to a consistent format.

# LLM Integration Approach Part 6

1. Outside of this, we also used the LLM to recommend us visualizations for specific things we wanted to visualize. An example of this can be seen below:
2. `chat$chat("Please recommend an appropriate visualization with a short explanation as to why it would be appropriate to answer the following question: Do views translate into engagement and interaction? (Does more views cause an increase in likes and comments?)")`

`**Visualization:** **Scatter Plot (Views vs. Likes & Views vs. Comments)**`

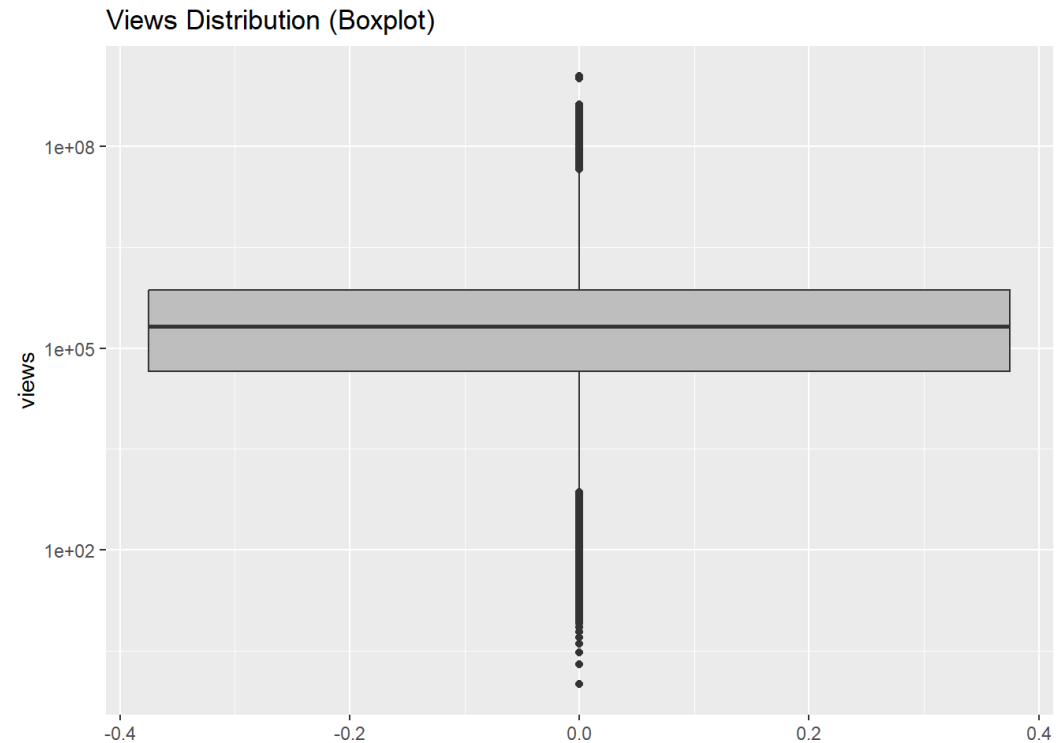
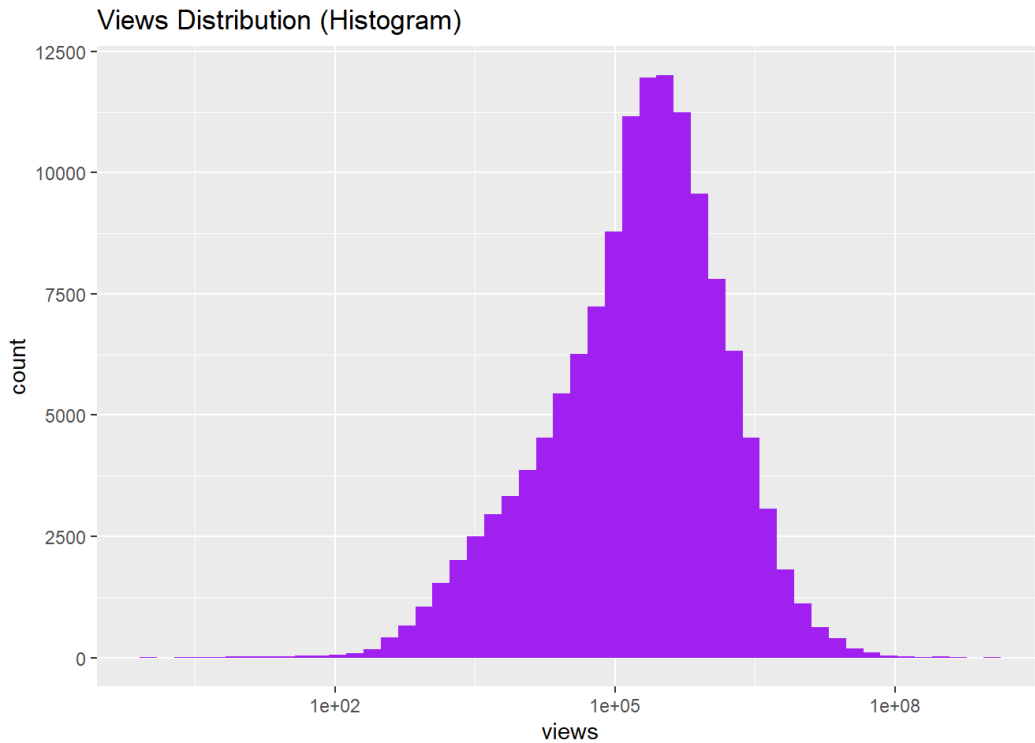
`**Explanation:** A scatter plot is the most direct and effective visualization to address this question. Each dot on the plot represents a single video. By plotting "Views" on the x-axis and "Likes" (or "Comments") on the y-axis, we can visually assess if there's a positive relationship: do videos with more views generally have more likes/comments? A rising trendline would suggest that views do indeed translate into engagement. It's a simple, easily understandable way to observe potential correlations between these key metrics.`

`It's the baseline for exploring the question because it directly displays the relationship between the variables we're interested in.`

# LLM Integration Approach Conclusion

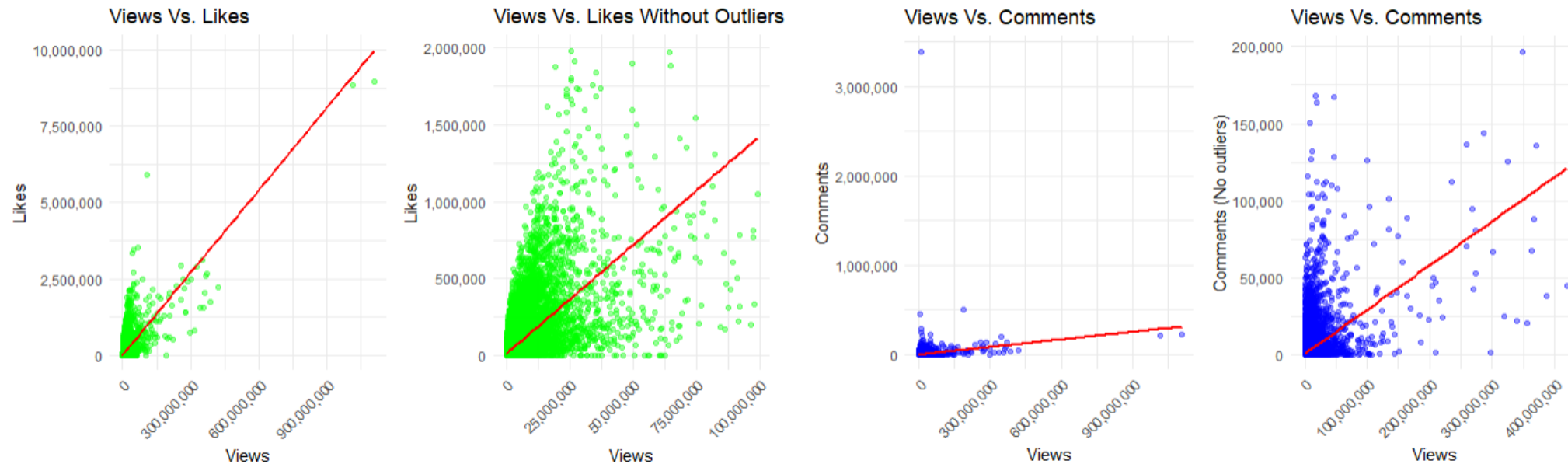
1. Overall, this LLM approach was the most useful for our needs. As mentioned in the related works slide, LLMs fall short as they can hallucinate. Due to this, directly automating data wrangling could cause significant problems. Therefore, we opted for this recommendation-querying approach instead.
2. For this approach, the main outcome was reduced time spent on visualizing the data. We would have spent more time figuring out the exact visualizations needed to answer our exploratory questions.
3. Furthermore, validating the LLMs output was simple, as we were the ones coding the visuals. Therefore, each one was manually confirmed and changed as necessary when creating them.

# Visualizations: Views Distributions



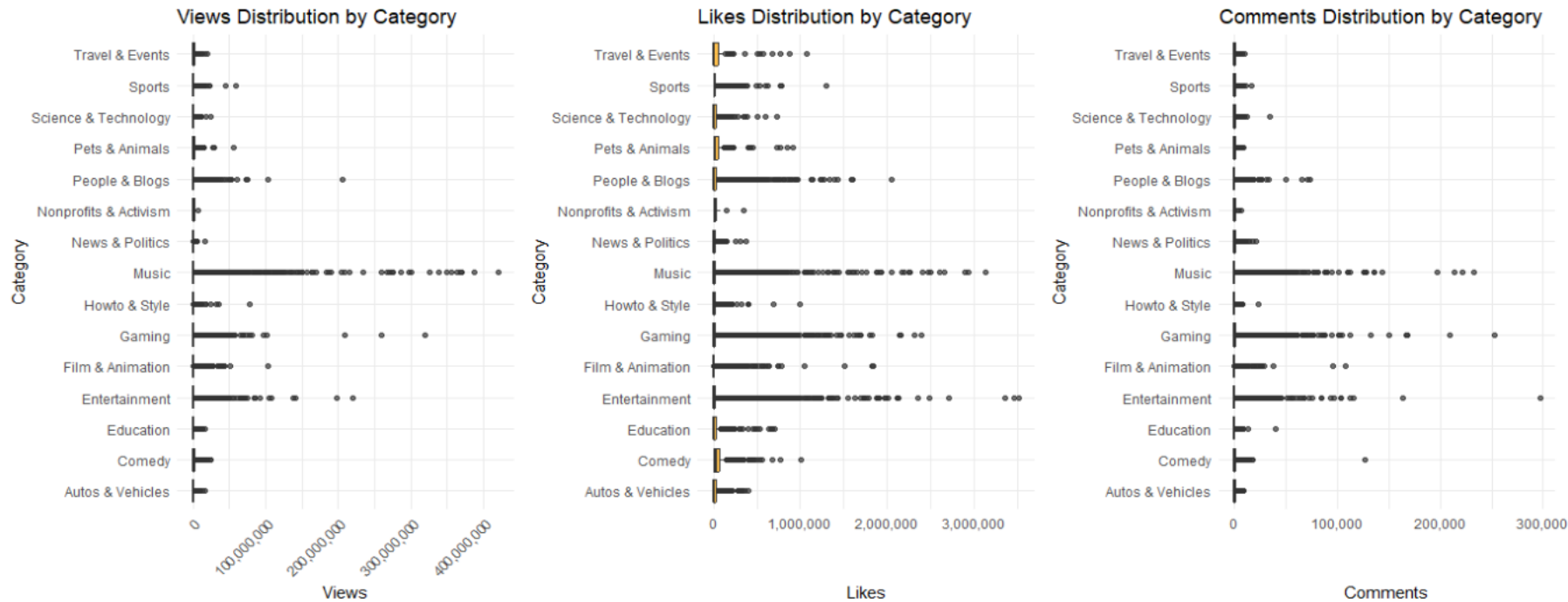
Our distribution of views shows that most trending videos have over 100,000 views. This makes sense logically as popular videos should be highly viewed.

# Does more views cause an increase in likes and comments?



Second graph in each is without major outliers. As expected, both likes and comments tend to increase as views increase.

# Does category effect engagement (likes), interaction (comments), and overall viewership?

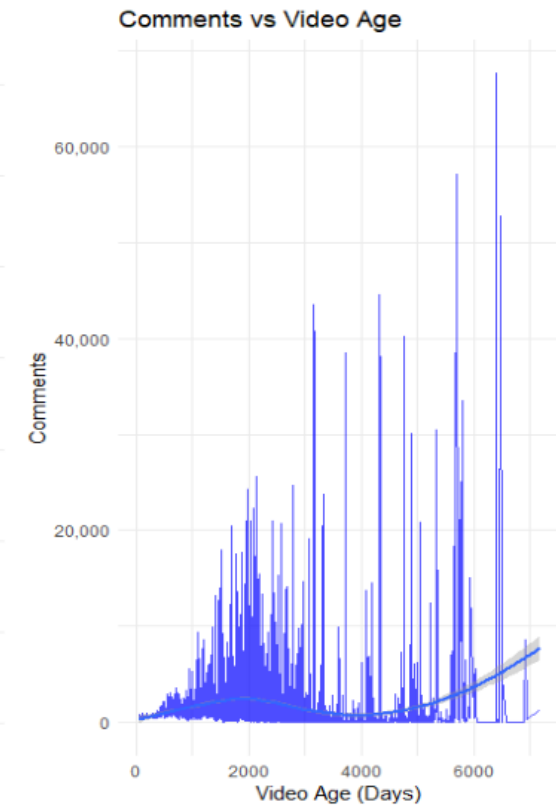
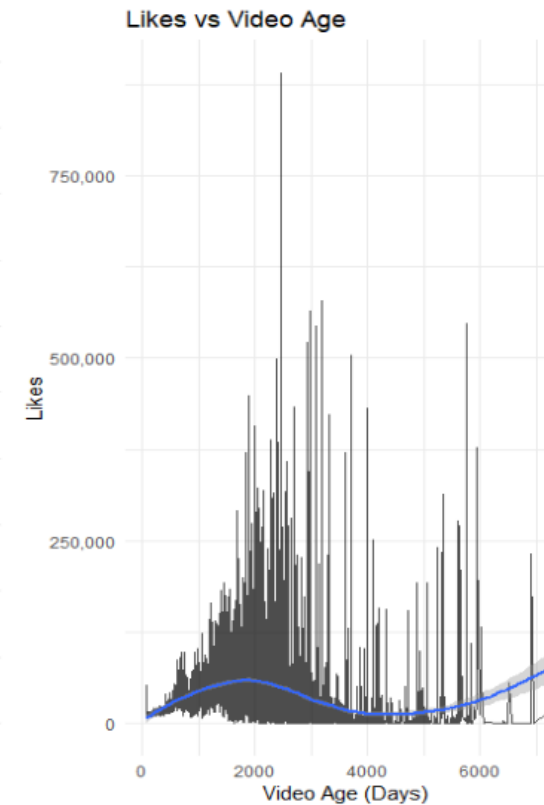
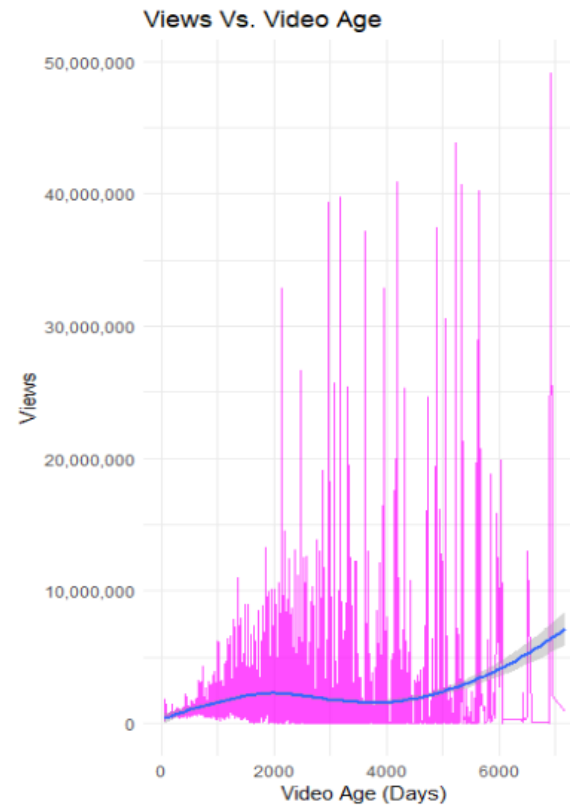


Category-wise, a huge number of trending videos tend to be in the music, gaming, entertainment, and People & Blogs section. Following that, each of those categories tend to have more videos with higher views, likes, and comments.

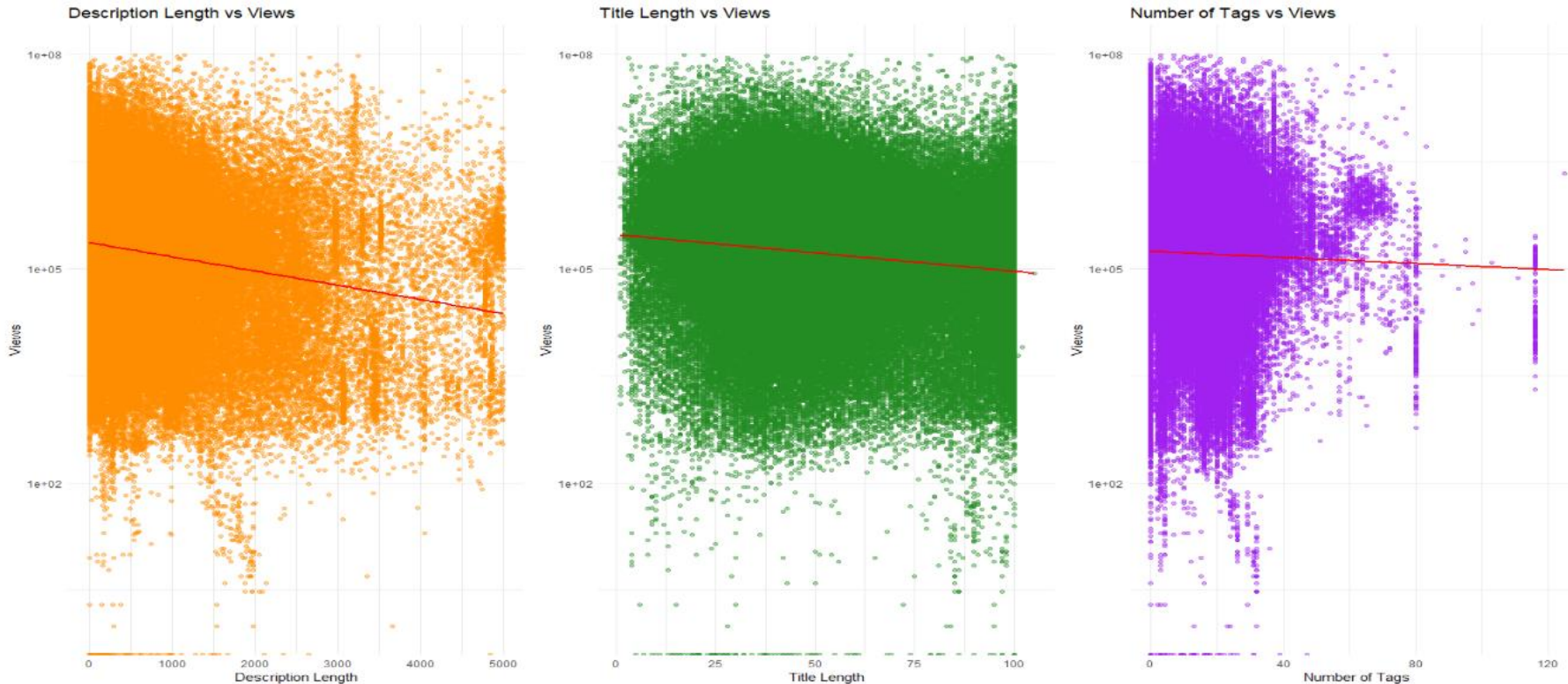


# Do newer videos tend to be more engaging and interactive or less?

The correlation between video age and views, likes, and comments is deceptive. One would think that higher age = higher video stats. However, this is not necessarily always true. For example, YouTube as a platform has grown over time, causing shifts on initial viewership and the overall culture of YouTube.



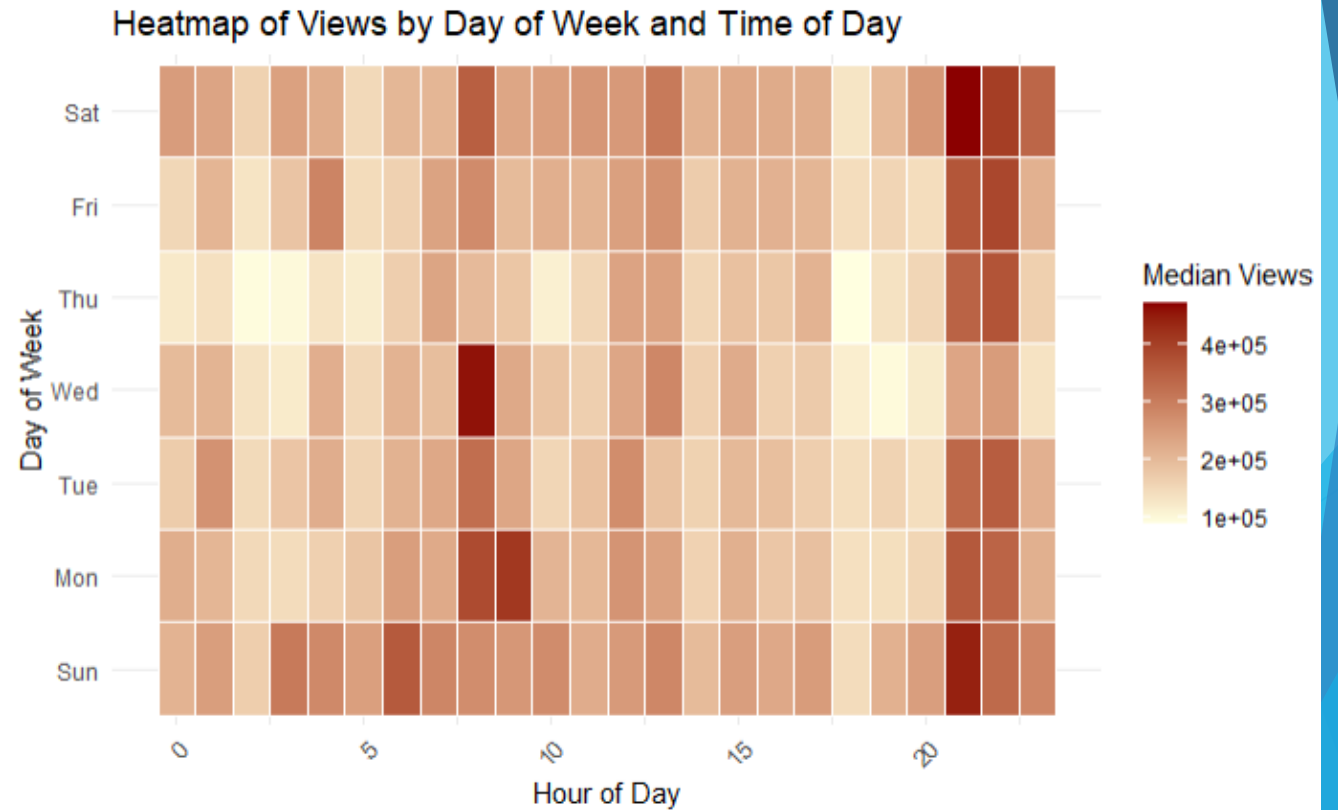
# Does tag amount and description/title length matter?



For the most part, it seems as though all three of these slightly negatively impact view performance across all videos.

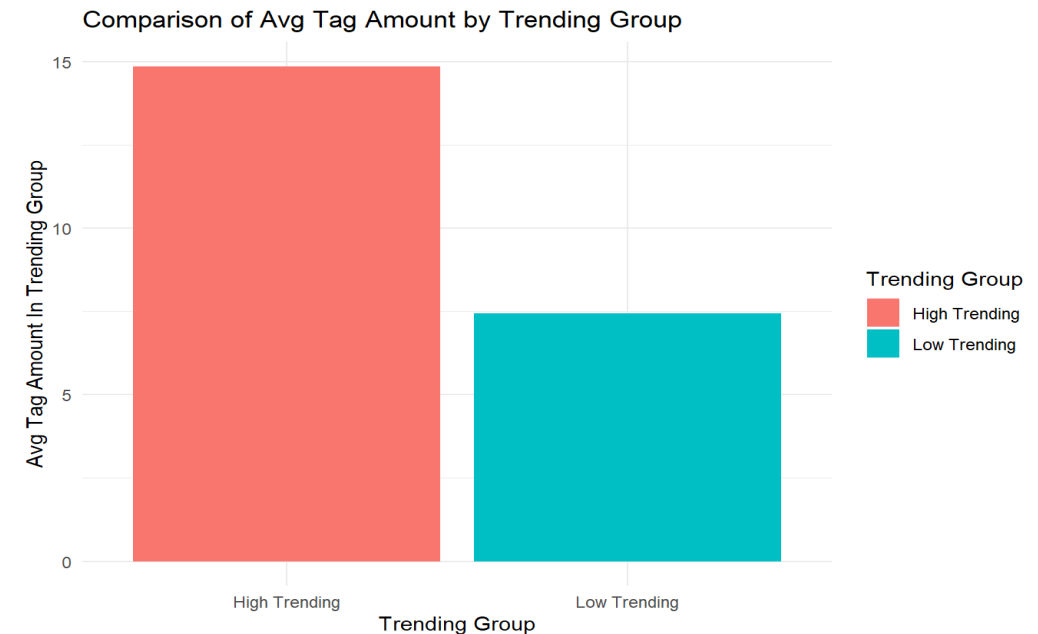
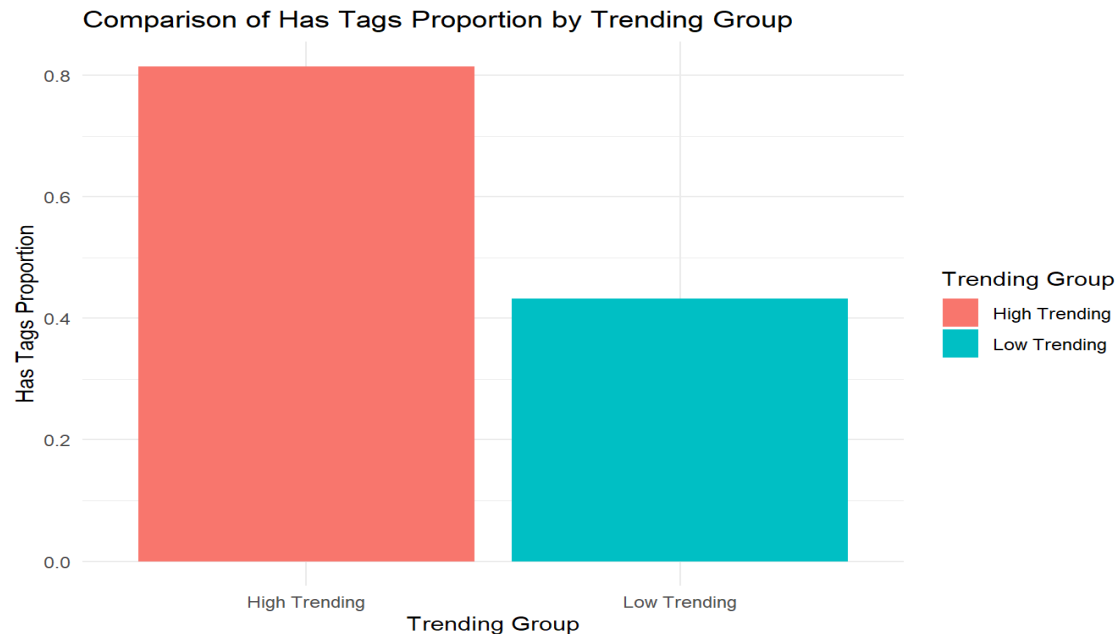
# Does time of day/week affect viewer numbers?

This shows how many views a video tends to have when they were published on a specific time or specific day. The most obvious part is that videos in the 21 and 22 hours of the day tend to have high median views for all days of the week.



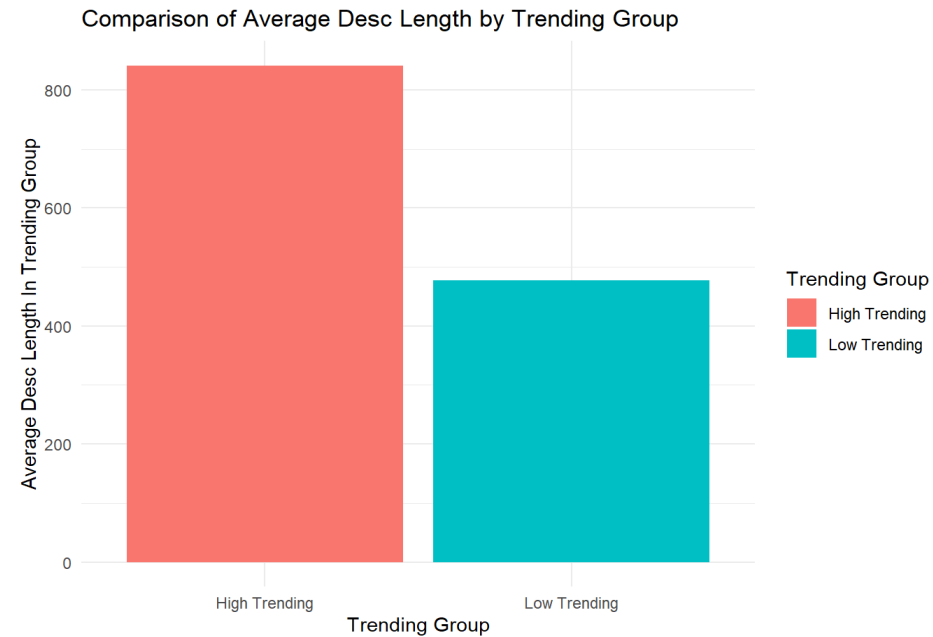
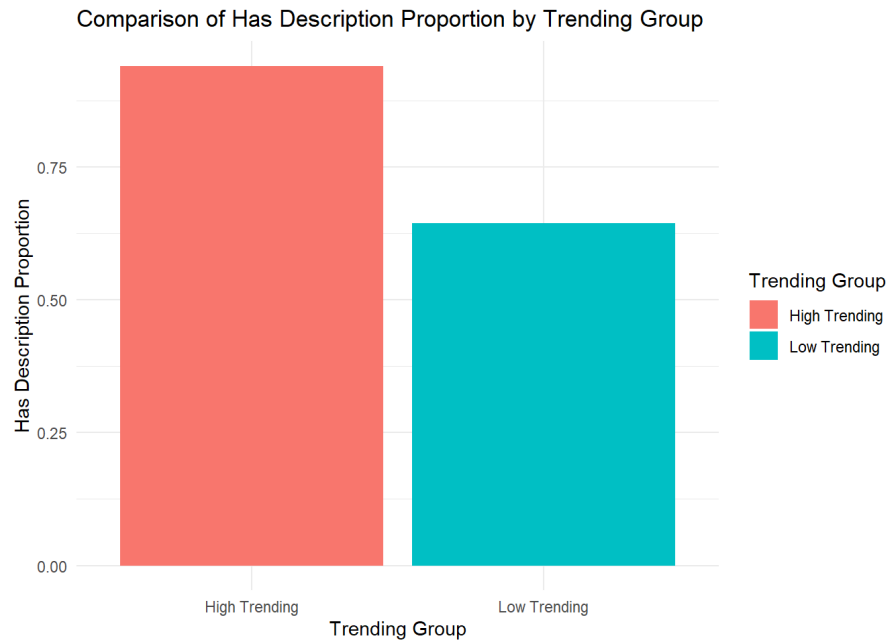
# Do channels with a lot of trending videos do anything different? (tags)

For these graphs, we can see that the top 1% of channels tend to more often include tags and have more tags than those not in the top 1%.



# Do channels with a lot of trending videos do anything different? (description)

Having a description and having a longer description follows the exact same pattern as the tag analysis. The top 1% tend to have descriptions more often and longer descriptions. However, this does contradict the previous analysis that overall longer descriptions and more tags decrease viewership.



# Future Work

1. Overall, there is much work that could be done and many directions one could take.
  - First, the LLM integration could be significantly better. A more powerful model obviously will help improve on hallucinations and overall recommendations.
    - While we did not have specific issues with hallucinations, the LLM tended to make extremely basic recommendations, not accounting for things like outliers. A higher quality model would in theory give more detailed explanations on what the visualization should look like.
    - Integrating using an API instead of local LLM usage would be a significant boost to the overall project.
    - A major point of which the LLM got wrong was exploring channel statistics. It told us to get the top 15 channels by viewership, then used those 15 channels and just reorder them by number of trending videos and average views per video. However, this is obviously incorrect, because effectively we are just looking at total views of the trended videos, not total views of the channel. Furthermore, it just reorders those 15 totals, not really looking at which channels dominated the most.
  - The data quality itself is high, but the types of data we need are simply not here.
    - For example, it would significantly help to have external data that gives channel statistics and non-trending video statistics. It is difficult to conclude whether certain channels dominate the trending charts. It is also difficult to conclude about what makes a trending video without analyzing videos that did not trend.
      1. Including high performing videos that did not trend, as well as channel statistics like subscriber count, total views, avg/median views, and more create a bigger picture about what makes trending videos trend.
    - Being able to compare YouTube trends to overall internet trends would also be a major boost.
    - Sentiment analysis and keyword extraction could be used to further analyze descriptions and titles and their effects on trends.

# Sources

- ▶ Google. (n.d.). *Trending charts on YouTube*. YouTube Help. <https://support.google.com/youtube/answer/7239739?hl=en#:~:text=Charts%20aim%20to%20balance%20all,uploads%20from%20the%20same%20channel>
- ▶ Zhou, W., Zhou, J., Wang, H., Li, Z., He, Q., Han, S., ... & Wu, F. (2026). *Can LLMs Clean Up Your Mess? A Survey of Application-Ready Data Preparation with LLMs*. arXiv preprint arXiv:2601.17058.
- ▶ Zhang, H., Dong, Y., Xiao, C., & Oyamada, M. (2023). *Large language models as data preprocessors*. arXiv preprint arXiv:2308.16361.
- ▶ Jaimovitch-López, G., Ferri, C., Hernández-Orallo, J., Martínez-Plumed, F., & Ramírez-Quintana, M. J. (2023). *Can language models automate data wrangling?*. Machine Learning, 112(6), 2053-2082.
- ▶ Ilya, S. (2026, February 26). *YouTube trending videos stats 2026*. Kaggle. <https://www.kaggle.com/datasets/bsthere/youtube-trending-videos-stats-2026>